

AI Track — Entry-Level Task

Evaluate and Compare Two Simple Models on the Same Dataset

Detail	Value
Assigned	Thursday, July 9, 2026
Due	Saturday, July 19, 2026 — 11:59 PM
Estimated effort	4–6 hours
Builds on	Session 3: Model evaluation, train/test splits, basic metrics
Pass bar	70 / 100

1. Objective

Train two simple classification models on the same dataset, evaluate both on a held-out test set using standard metrics, and write a short, evidence-based recommendation for which model you would use. This is the complete Session 3 workflow, end to end.

2. What you will learn

- Making a proper train/test split — and why you never evaluate on training data
- Training baseline models in scikit-learn
- Computing and interpreting accuracy, precision, recall, F1, and the confusion matrix
- Presenting a fair model comparison with a table, plots, and a written conclusion

3. Step 1 — Pick ONE dataset

Dataset	How to load	Difficulty
Breast Cancer (binary)	<code>load_breast_cancer()</code>	Easiest — recommended
Iris (3 classes)	<code>load_iris()</code>	Easy
Titanic survival	<code>sns.load_dataset("titanic")</code>	Needs light cleaning

4. Step 2 — Quick exploration

- Number of rows/columns, feature names, target classes
- Class balance (`value_counts()` plus a simple bar chart)
- Missing values (only Titanic needs handling — a simple drop or fill is fine)

5. Step 3 — Split ONCE

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

Both models must use this same split — otherwise the comparison is unfair.

6. Step 4 — Train two models

- **Model A:** `LogisticRegression(max_iter=1000)`
- **Model B:** `DecisionTreeClassifier(random_state=42)` — or `KNeighborsClassifier()`

Default parameters are fine. If you scale features, fit the scaler on **train only** (`fit_transform` on train, `transform` on test).

7. Step 5 — Evaluate on the TEST set only

For each model, report:

- Accuracy
- Precision, recall and F1 (`classification_report`)
- Confusion matrix as a plot (`ConfusionMatrixDisplay`)

Then build one comparison table, for example:

Metric	Logistic Regression	Decision Tree
Accuracy	0.965	0.930
Precision	...	...
Recall	...	...
F1-score	...	...

Numbers above are illustrative only — report your real results.

8. Step 6 — Visualize the comparison

At least two plots:

1. Side-by-side confusion matrices (one per model).
2. A grouped bar chart comparing the four metrics for both models.

9. Step 7 — Write your conclusion (150–300 words)

Answer, in your own words:

1. Which model performed better overall, and on which specific metrics?
2. Are the errors different in kind? (Look at the confusion matrices — more false negatives?)
3. For this problem, is precision or recall more important? Why?
4. Which model would you ship, and what would you try next to improve it?

10. Rules — breaking these means rework

Never call `.fit()` on anything using test data — scalers and encoders included. That is data leakage.

- Set `random_state=42` everywhere for reproducibility.
- The notebook must run cleanly top to bottom — do **Restart & Run All** before submitting.
- Every step needs a short markdown cell explaining what and why.

11. Bonus (optional, up to +10 pts)

- Run 5-fold cross-validation (`cross_val_score`) and compare with your single-split numbers.
- Add a third model (e.g. `RandomForestClassifier`).
- Tune one hyperparameter (e.g. `tree max_depth`) — never on the test set.

12. Deliverables

1. `model_comparison_<name>.ipynb` — clean, commented, runs top to bottom without errors.
2. A final summary cell — dataset, the two models, key numbers, and your recommendation in 3–5 sentences.
3. Optional: `requirements.txt` with the libraries you used.

13. How to submit

Push the notebook to GitHub, or share a Google Colab link with viewer access, in the ai-track channel (or to your mentor) before the deadline. If you will be late, tell your mentor before the due date, not after. Written feedback within 2 working days.

14. Grading rubric (100 pts)

Criterion	Points
Loading, brief exploration & clean split (no leakage)	20
Both models trained on the same split	15
Metrics on the test set + comparison table	25
Visualizations (confusion matrices + metric chart)	15
Written conclusion: interpretation & reasoning	15
Reproducibility, code quality & markdown	10
Bonus	+10
Total (pass bar 70)	100

15. Common mistakes

- Accuracy can lie on imbalanced data — always check per-class recall.
- Comparing models trained on different splits is meaningless — one split, two models.
- `classification_report` gives precision, recall & F1 per class in one call.
- A default decision tree often overfits — if train accuracy test accuracy, say so.

16. Resources

- scikit-learn — `train_test_split`: scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html
- scikit-learn — Model evaluation guide: scikit-learn.org/stable/modules/model_evaluation.html
- scikit-learn — `ConfusionMatrixDisplay`: scikit-learn.org/stable/modules/generated/sklearn.metrics.ConfusionMatrixDisplay.html

- Google ML Crash Course — Classification: developers.google.com/machine-learning/crash-course/classification

Honesty rule. *You may read documentation and search freely. You may not copy a complete solution. If AI tools are permitted, you must be able to explain every line you submit.*